

M12 Teaching Book

Distributed Processing with Spark / PySpark - beginner-first, self-contained teaching

How to use this book

Watch the assigned visual first, then close it. Use this book as the authoritative course explanation and current Spark docs for exact modern behavior. Reproduce the guided example, close help, attempt independently, validate controls, explain back, and only then use protected Review if needed.

Current baseline

Apache Spark/PySpark baseline: 4.2.0. PySpark supports Python 3.10+; Java 17+ is required. Course laptop profile: local[2], driver memory around 2g, and shuffle partitions 4 for tiny teaching data. Spark defaults such as 200 shuffle partitions or 10 MB auto-broadcast threshold are taught as defaults, not blindly copied into production.

Boundary

M12 teaches distributed Spark execution and Parquet I/O. M13 owns deeper Delta Lake runtime. M14 owns Kafka/streaming. No Delta or Kafka install is required here.

DE12-01 - Why Spark exists: driver, executors, partitions and tasks

Prerequisite: M11 mastered and G04 passed.

1. Outcome

Explain Spark as a distributed execution engine and distinguish driver, executors, partitions, jobs, stages and tasks.

2. Teacher explanation

Spark exists because one process and one machine eventually become a bottleneck. The driver builds the application plan and coordinates work. Executors run tasks and hold data/cache for that application. A partition is a unit of distributed data work; one task typically works on one partition for a stage. An action creates a job. Shuffle boundaries can split a job into stages. These are execution concepts, not business grain: a dataset may still be one row per order whether it has 2 partitions or 2,000.

3. Working demonstration

Sketch 16 orders split across four boxes. Mark one driver that plans a groupBy and four task boxes that process partitions. Then draw a shuffle before the final aggregate.

4. Why this matters

Distributed systems can make wrong work faster. The engineer must preserve grain, cardinality, measures and deterministic behavior while also reasoning about partitions, movement, memory and plans.

Video first

[DataTalksClub Zoomcamp - Apache Spark / PySpark](#)

Whole lesson; use for driver/executor/partition/task mental model.

DE12-01 - Practice, Proof & Transfer

5. Guided reproduction

Run the partition/skew simulator. Use it only to visualize uneven buckets; explicitly state that it is not Spark.

6. Independent task

Draw the driver/executor/partition/job/stage/task relationship from memory and explain what changes when partition count changes.

7. Validate before trusting

Your explanation keeps business row grain separate from physical partitions and says executors execute tasks while the driver coordinates the application.

8. Recover if weak

If everything feels like “a server,” start with one question: who plans, who executes, and what chunk of data does one task receive?

9. Explain in your own words

Why can changing partition count change performance without changing the correct business answer?

10. Evidence to save

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran.

11. Protected Review rule

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

Authoritative current references

[Spark Cluster Mode Overview](#)

[Apache Spark 4.2.0 release](#)

DE12-02 - PySpark setup, SparkSession and laptop-safe local mode

Prerequisite: DE12-01.

1. Outcome

Install/verify PySpark 4.2.0 safely and create a laptop-safe local SparkSession without treating local mode as a real cluster.

2. Teacher explanation

PySpark is the Python API for Spark. For this course, first use is here in M12. Spark 4.2.0 supports Python 3.10+ and needs Java 17+. Local mode runs Spark components on your computer so you can learn the API and execution model, but `local[2]` is not a production cluster. On an 8 GB learner laptop, use two local worker threads, a small shuffle-partition count, and about 2 GB driver memory. Close Docker, Kafka and other heavy tools while practicing Spark.

3. Working demonstration

Verify `java -version`; then `python -m pip install "pyspark==4.2.0"`; verify `python -c "import pyspark; print(pyspark.\_\_version\_\_)"`; start a SparkSession with local[2], 2g driver memory and 4 shuffle partitions.

4. Why this matters

Distributed systems can make wrong work faster. The engineer must preserve grain, cardinality, measures and deterministic behavior while also reasoning about partitions, movement, memory and plans.

Video first

[DataTalksClub Zoomcamp - Apache Spark / PySpark](#)

Setup/orientation visual; current Spark 4.2.0 install guidance below controls the learner setup.

DE12-02 - Practice, Proof & Transfer

5. Guided reproduction

Create a session named m12-check, print Spark version/master/shuffle partitions, then stop it cleanly.

6. Independent task

Recreate the session from a blank file and explain every builder setting.

7. Validate before trusting

Expected: Spark version 4.2.0, local[2], shuffle partitions 4. Exact JVM/OS text may differ.

8. Recover if weak

If Java errors appear, verify Java ≥ 17 and JAVA_HOME before reinstalling Spark. If import fails, check that ``python -m pip`` and ``python`` are the same environment.

9. Explain in your own words

Why do we deliberately use local[2] and 4 shuffle partitions in a teaching lab even though Spark defaults differ?

10. Evidence to save

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran.

11. Protected Review rule

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

Authoritative current references

[PySpark 4.2.0 Installation](#)

[PySpark 4.2.0 Getting Started](#)

DE12-03 - DataFrames and explicit schemas

Prerequisite: DE12-02.

1. Outcome

Build DataFrames with explicit schemas and explain why production ingestion should not blindly trust inference.

2. Teacher explanation

A Spark DataFrame is distributed tabular data with named columns and types plus a query plan. Spark can infer CSV types, but inference costs work and can make contracts depend on sample data. An explicit StructType makes required types visible and repeatable. Nullable is part of the schema contract. Schema correctness is separate from business validity: a positive amount rule still needs a data-quality check.

3. Working demonstration

Use the provided order StructType in m12_reference_pipeline.py and read the normal orders.csv with header=true and no inferSchema. PrintSchema before any transformation.

4. Why this matters

Distributed systems can make wrong work faster. The engineer must preserve grain, cardinality, measures and deterministic behavior while also reasoning about partitions, movement, memory and plans.

Video first

[DataTalksClub Zoomcamp - Apache Spark / PySpark](#)

DataFrame visual; use the relevant schema/DataFrame section.

DE12-03 - Practice, Proof & Transfer

5. Guided reproduction

Create an explicit schema for the retest fixture, then identify which field is a business-validity flag rather than a type rule.

6. Independent task

Close the guided code. From a blank file, define a schema for a fresh six-column shipment fixture, load it without inferSchema, and prove row count, types and one business-validity rule independently.

7. Validate before trusting

Normal orders: 16 rows, quantity 31, amount 3770.00. order_id/customer_id/region/category are strings; qty integer; amount double.

8. Recover if weak

If numeric columns arrive as strings, check whether an explicit schema was applied and whether malformed data was silently turned null. Inspect bad records rather than casting blindly.

9. Explain in your own words

Why is an explicit schema helpful but still insufficient as a full data contract?

10. Evidence to save

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran.

11. Protected Review rule

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

Authoritative current references

[PySpark DataFrame quickstart](#)

[DataFrame API](#)

DE12-04 - Spark SQL and expression equivalence

Prerequisite: DE12-03.

1. Outcome

Prove equivalent results through DataFrame expressions and Spark SQL and explain when each style improves maintainability.

2. Teacher explanation

Spark SQL and the DataFrame API both describe logical transformations that Spark optimizes. Registering a DataFrame as a temp view lets SQL express the same grouped result as `groupBy().agg()`. Equivalence should be proved by result equality, not assumed because both queries look sensible. Choose style based on team readability, composability and testability; do not claim one is universally faster when they compile to equivalent plans.

3. Working demonstration

In the reference pipeline, compare the region aggregate built with `groupBy/agg` against the SQL query. Sort both before equality checking because distributed row order is not guaranteed without `ORDER BY`.

4. Why this matters

Distributed systems can make wrong work faster. The engineer must preserve grain, cardinality, measures and deterministic behavior while also reasoning about partitions, movement, memory and plans.

Video first

[DataTalksClub Zoomcamp - Apache Spark / PySpark](#)

Spark SQL visual; compare SQL and DataFrame-expression routes.

DE12-04 - Practice, Proof & Transfer

5. Guided reproduction

Write the category total once with DataFrame functions and once with SQL, then compare sorted results.

6. Independent task

Close both examples. Rebuild a two-column grouped result once with DataFrame expressions and once with Spark SQL on a changed category/zone grouping, normalize order, and prove exact equality.

7. Validate before trusting

Both routes must have three category rows and total amount 3770.00.

8. Recover if weak

If equality fails, compare filters, null behavior, aggregation aliases and ordering. Do not blame Spark SQL before comparing logical expressions.

9. Explain in your own words

Why must you sort or otherwise normalize distributed results before using row-order-sensitive equality?

10. Evidence to save

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran.

11. Protected Review rule

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

Authoritative current references

[Spark SQL programming guide](#)

DE12-05 - Lazy transformations and actions

Prerequisite: DE12-04.

1. Outcome

Predict lazy execution and identify which operations are transformations versus actions.

2. Teacher explanation

Spark transformations describe new datasets without necessarily executing them immediately. Actions demand a result or side effect and trigger a job. Lazy planning lets Spark optimize across a chain before execution. ``select``, ``filter``, ``withColumn``, joins and `groupBy` expressions are transformations; ``count``, ``collect``, ``show``, many writes and similar result-producing calls trigger work. Reusing an unpersisted lineage can rerun computation on each action.

3. Working demonstration

Trace ``orders.filter(...).select(...).groupBy(...).agg(...)`` and mark where no result is demanded. Then mark ``count()`` as the action that triggers execution.

4. Why this matters

Distributed systems can make wrong work faster. The engineer must preserve grain, cardinality, measures and deterministic behavior while also reasoning about partitions, movement, memory and plans.

Video first

[DataTalksClub Zoomcamp - Apache Spark / PySpark](#)

Transformations/actions and first-look execution context.

DE12-05 - Practice, Proof & Transfer

5. Guided reproduction

Write a five-step lineage with exactly one final action and predict when data is actually read.

6. Independent task

From memory, write a new lazy lineage with at least four transformations and exactly two actions. Before running it, predict which lines trigger jobs and which only extend the plan; then verify on the learner runtime.

7. Validate before trusting

Your classification must explain why printing a DataFrame object is not the same as materializing all its rows.

8. Recover if weak

If you are unsure, ask whether the call returns a new logical DataFrame or asks Spark to compute/return/write a result.

9. Explain in your own words

How does lazy execution help optimization, and how can repeated actions surprise a beginner?

10. Evidence to save

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran.

11. Protected Review rule

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

Authoritative current references

[RDD programming guide - transformations/actions](#)

DE12-06 - Jobs, stages, tasks and physical plans

Prerequisite: DE12-05.

1. Outcome

Connect actions to jobs/stages/tasks and use EXPLAIN/UI evidence to reason about the physical plan.

2. Teacher explanation

A logical query says what result is wanted. Spark produces optimized logical and physical plans describing how to execute it. An action creates one or more jobs; shuffle boundaries often create stage boundaries; tasks execute per partition. ``explain(mode="formatted")`` exposes plan operators without requiring you to guess. The Spark UI then shows runtime jobs, stages, tasks and metrics. Plan reading is evidence: identify scans, exchanges/shuffles, join type and aggregation operators before tuning.

3. Working demonstration

Call ``explain(mode="formatted")`` on a grouped DataFrame before the action. During a learner-machine run, open the Spark UI and relate the action to job/stage/task counts.

4. Why this matters

Distributed systems can make wrong work faster. The engineer must preserve grain, cardinality, measures and deterministic behavior while also reasoning about partitions, movement, memory and plans.

Video first

[DataTalksClub Zoomcamp - Apache Spark / PySpark](#)

Jobs/stages/tasks/execution-plan visual.

DE12-06 - Practice, Proof & Transfer

5. Guided reproduction

Save one plan and annotate Scan, Exchange, Aggregate and any join node you see.

6. Independent task

Take a changed aggregation/join pipeline, save formatted EXPLAIN output, annotate scan, filter, exchange, join and aggregate nodes that actually appear, and connect one action to the observed job/stage/task evidence in the Spark UI.

7. Validate before trusting

You can explain at least one cause of an Exchange and distinguish plan evidence from a tuning guess.

8. Recover if weak

If the plan is overwhelming, find the data source first, then work upward to filters, joins/exchanges and aggregates.

9. Explain in your own words

What evidence would convince you that a shuffle happened instead of merely assuming groupBy is expensive?

10. Evidence to save

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran.

11. Protected Review rule

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

Authoritative current references

[Spark Web UI](#)

[Spark SQL EXPLAIN](#)

DE12-07 - repartition vs coalesce

Prerequisite: DE12-06.

1. Outcome

Choose repartition or coalesce deliberately and explain the shuffle/cost trade-off.

2. Teacher explanation

``repartition(n)`` can increase or decrease partitions and normally performs a shuffle to redistribute data. ``coalesce(n)`` is commonly used to reduce partitions without a full shuffle, trading balanced distribution for lower movement. Neither changes the business rows. Repartitioning by a key can improve downstream distribution but can also expose skew. Never use `repartition(1)` as a universal fix for small files: it can bottleneck work on one partition.

3. Working demonstration

On the normal DataFrame, record `getNumPartitions()`, then compare `repartition(4)`, `repartition(4,"region")` and `coalesce(1)` plans/partition counts.

4. Why this matters

Distributed systems can make wrong work faster. The engineer must preserve grain, cardinality, measures and deterministic behavior while also reasoning about partitions, movement, memory and plans.

Video first

[DataTalksClub Zoomcamp - Apache Spark / PySpark](#)

Partitions and shuffle context; book controls exact repartition/coalesce reasoning.

DE12-07 - Practice, Proof & Transfer

5. Guided reproduction

Write a decision table for increasing parallelism, reducing output files, key distribution and avoiding a single-partition bottleneck.

6. Independent task

On changed data, choose between repartition and coalesce for two separate goals: balanced keyed work and fewer output files. Record before/after partition counts and defend why neither operation changes business grain.

7. Validate before trusting

Every choice includes its intended downstream effect and a correctness control that row count/amount stay 16/3770.00.

8. Recover if weak

If you only remember "coalesce is faster," state whether you are increasing or decreasing partitions and whether balanced redistribution is required.

9. Explain in your own words

When is avoiding a shuffle with coalesce a bad trade because the remaining partitions are badly imbalanced?

10. Evidence to save

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran.

11. Protected Review rule

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

Authoritative current references

[DataFrame.repartition](#)

[DataFrame.coalesce](#)

DE12-08 - Aggregations and shuffle cost

Prerequisite: DE12-07.

1. Outcome

Measure aggregation shuffle cost and choose partition counts using evidence rather than copying defaults.

2. Teacher explanation

Aggregations by key require records with the same key to meet, which commonly causes shuffle. Shuffle moves/serializes data and can create many intermediate blocks. `spark.sql.shuffle.partitions` defaults to 200 in Spark 4.2.0, which is intentionally too many for this tiny course dataset; the lab uses 4 to keep local work visible and light. Production values depend on data size, cluster cores, skew and AQE. Correctness comes first: tuning must preserve grouped totals.`

3. Working demonstration

Group orders by region and sum amount. Inspect the plan for Exchange. Compare the tiny lab with shuffle partitions 4 versus a deliberately larger value and observe task/partition differences on your machine.

4. Why this matters

Distributed systems can make wrong work faster. The engineer must preserve grain, cardinality, measures and deterministic behavior while also reasoning about partitions, movement, memory and plans.

Video first

[DataTalksClub Zoomcamp - Apache Spark / PySpark](#)

Aggregation/shuffle visual.

DE12-08 - Practice, Proof & Transfer

5. Guided reproduction

Explain why “200 is the Spark default” does not mean “200 is correct for every dataset.”

6. Independent task

Build a changed two-key aggregation, reconcile grouped totals back to source totals, then inspect the plan for Exchange/shuffle evidence. Explain why correctness proof must come before performance tuning.

7. Validate before trusting

Region totals must sum back to 3770.00 and grouped row count is four.

8. Recover if weak

If tuning discussion becomes guesswork, return to input size, partition sizes, task count, shuffle bytes and runtime evidence.

9. Explain in your own words

Why can too many tiny shuffle partitions be inefficient even though more partitions sound more parallel?

10. Evidence to save

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran.

11. Protected Review rule

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

Authoritative current references

[Spark SQL performance tuning](#)

DE12-09 - Joins and broadcast strategy

Prerequisite: DE12-08.

1. Outcome

Choose join strategy with size/statistics awareness and use broadcast only when the small side is genuinely safe.

2. Teacher explanation

Large joins often shuffle data so matching keys meet. If one side is small, Spark can broadcast it to executors and avoid shuffling the larger side. Spark 4.2.0 has a 10 MB automatic broadcast threshold by default for suitable plans, but hints are not unconditional guarantees. Broadcast is a performance strategy, not a correctness fix. First prove join keys and cardinality. Broadcasting an unexpectedly large table can create memory pressure.

3. Working demonstration

Join 16 orders to the 13-row customer dimension with `broadcast(customers)`. Inspect the formatted plan and look for the broadcast join operator on your learner runtime.

4. Why this matters

Distributed systems can make wrong work faster. The engineer must preserve grain, cardinality, measures and deterministic behavior while also reasoning about partitions, movement, memory and plans.

Video first

[DataTalksClub Zoomcamp - Apache Spark / PySpark](#)

Join visual; book adds current broadcast rules and correctness proof.

DE12-09 - Practice, Proof & Transfer

5. Guided reproduction

Remove the explicit hint and compare the plan. Explain what statistics/threshold information influences automatic broadcast.

6. Independent task

Join a changed fact fixture to a deliberately small unique dimension. Prove dimension-key uniqueness, compare an ordinary join with an explicit broadcast-hint plan, and save row/amount controls showing both are semantically identical.

7. Validate before trusting

Correct join output remains 16 rows, amount 3770.00, and zero unmatched customer segments.

8. Recover if weak

If a broadcast hint looks ignored, check join type, plan, table size/statistics and Spark support before forcing another hint.

9. Explain in your own words

Why should you prove key uniqueness and row-count invariants before celebrating a faster broadcast join?

10. Evidence to save

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran.

11. Protected Review rule

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

Authoritative current references

[Spark SQL performance tuning - join strategy](#)

[functions.broadcast](#)

DE12-10 - Join correctness, fan-out and reconciliation

Prerequisite: DE12-09.

1. Outcome

Detect fan-out and reconcile joins using row-count, key and measure controls.

2. Teacher explanation

A join can run successfully and still multiply facts. If a supposed one-side dimension contains duplicate join keys, each matching fact can be duplicated. DISTINCT after the join hides symptoms and may delete legitimate repeated facts. Correct repair begins by stating source grain and expected cardinality, testing one-side uniqueness, and reconciling row count plus a stable measure. The provided simulator intentionally duplicates customer C02 to make this visible without Spark.

3. Working demonstration

Run `join_fanout_simulator.py`. Compare good and bad joins. Identify the duplicate dimension key and the inflated row/amount controls.

4. Why this matters

Distributed systems can make wrong work faster. The engineer must preserve grain, cardinality, measures and deterministic behavior while also reasoning about partitions, movement, memory and plans.

Video first

[DataTalksClub Zoomcamp - Apache Spark / PySpark](#)

Join visual plus course-owned fan-out/reconciliation task.

DE12-10 - Practice, Proof & Transfer

5. Guided reproduction

In PySpark, load `customers_with_duplicate.csv`, `groupBy(customer_id).count().filter(count>1)`, then prove the join inflates before repairing the dimension contract.

6. Independent task

Create a tiny duplicate-key dimension that causes fan-out. Demonstrate the wrong joined row count/measure, identify the duplicated key, repair the dimension contract, rerun, and prove the original fact totals are restored.

7. Validate before trusting

Good join: 16 rows / 3770.00. The bad dimension must produce more rows and a larger amount than the source.

8. Recover if weak

If a join has extra rows, do not add `DISTINCT` first. Test key uniqueness on each side and locate the key(s) with multiplicity.

9. Explain in your own words

Why is ``DISTINCT`` a dangerous response to a fan-out join?

10. Evidence to save

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran.

11. Protected Review rule

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

Authoritative current references

[DataFrame.join](#)

DE12-11 - Data skew

Prerequisite: DE12-10.

1. Outcome

Measure skew and explain why a few hot keys can dominate stage time.

2. Teacher explanation

Skew means work is unevenly distributed: one or a few keys/partitions contain much more data than others. A job can appear mostly complete while one straggler task runs much longer. Measure key-frequency distribution and partition metrics before applying techniques such as salting or AQE skew handling. Salting adds complexity and must preserve join/group semantics, so it is not a default beginner trick.

3. Working demonstration

Use the simulator region counts as a small visual. Then on Spark `groupBy(region).count()` and inspect max/min counts. The tiny course data is not a performance benchmark; it only teaches the diagnostic pattern.

4. Why this matters

Distributed systems can make wrong work faster. The engineer must preserve grain, cardinality, measures and deterministic behavior while also reasoning about partitions, movement, memory and plans.

Video first

[DataTalksClub Zoomcamp - Apache Spark / PySpark](#)

Shuffle/skew visual context; book adds explicit skew measurement.

DE12-11 - Practice, Proof & Transfer

5. Guided reproduction

Create a changed in-memory DataFrame with one key repeated far more than others. Predict which partition/task becomes a straggler after hash redistribution.

6. Independent task

Create a changed skewed-key fixture, calculate key-frequency concentration, predict the straggler key/partition risk, and propose one mitigation only after showing measured skew evidence.

7. Validate before trusting

Your evidence includes key-frequency counts and an explanation of why equal partition count does not guarantee equal partition size.

8. Recover if weak

If you cannot see skew, sort key counts descending and compare the largest key with median/typical keys.

9. Explain in your own words

How can 100 partitions still perform poorly if one key owns 70% of all records?

10. Evidence to save

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran.

11. Protected Review rule

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

Authoritative current references

[Spark SQL performance tuning - skew](#)

DE12-12 - Adaptive Query Execution (AQE)

Prerequisite: DE12-11.

1. Outcome

Explain AQE and verify which adaptive features are enabled instead of treating it as magic auto-tuning.

2. Teacher explanation

Adaptive Query Execution uses runtime statistics to revise parts of a SQL physical plan while the query runs. Spark 4.2.0 documents AQE as enabled by default. It can coalesce post-shuffle partitions, handle skewed shuffle partitions, and sometimes change join strategy. AQE does not make bad joins correct and does not remove the need for sane initial configuration or validation. Compare initial/final plans where available and keep business controls invariant.

3. Working demonstration

Print `spark.sql.adaptive.enabled` and inspect a grouped/join plan. On a sufficiently meaningful learner dataset, compare the adaptive/final plan after an action.

4. Why this matters

Distributed systems can make wrong work faster. The engineer must preserve grain, cardinality, measures and deterministic behavior while also reasoning about partitions, movement, memory and plans.

Video first

[DataTalksClub Zoomcamp - Apache Spark / PySpark](#)

Execution-plan visual; current Spark docs control AQE semantics.

DE12-12 - Practice, Proof & Transfer

5. Guided reproduction

Write a three-column note: static plan assumption -> runtime fact -> possible AQE adaptation.

6. Independent task

On a changed join+aggregate query, record the AQE configuration, capture the initial/adaptive plan evidence available after an action, and state which adaptation Spark may make without claiming AQE fixes incorrect joins.

7. Validate before trusting

Your explanation names at least two AQE behaviors and says correctness controls remain unchanged.

8. Recover if weak

If you describe AQE as “Spark automatically makes everything fast,” return to the exact documented adaptive rules and their configuration boundaries.

9. Explain in your own words

What kinds of decisions can AQE revise, and what important problems can it not fix for you?

10. Evidence to save

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran.

11. Protected Review rule

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

Authoritative current references

[Spark SQL performance tuning - AQE](#)

DE12-13 - Caching and persistence

Prerequisite: DE12-12.

1. Outcome

Use cache/persist only for reused expensive data and prove unpersist lifecycle.

2. Teacher explanation

Caching stores computed data so later actions can reuse it instead of rebuilding the same lineage. Cache has a memory/storage cost and can become stale relative to a changed external source. Cache when a reused dataset is expensive enough to justify storage; materialize it with an action, compare repeated-use evidence, and `unpersist()` when done. Do not cache every DataFrame.

3. Working demonstration

Create a filtered/derived DataFrame used by two actions. Cache it, materialize with count, inspect storage/UI, run the second action, then unpersist.

4. Why this matters

Distributed systems can make wrong work faster. The engineer must preserve grain, cardinality, measures and deterministic behavior while also reasoning about partitions, movement, memory and plans.

Video first

[DataTalksClub Zoomcamp - Apache Spark / PySpark](#)

Execution/storage context; book adds cache/unpersist proof.

DE12-13 - Practice, Proof & Transfer

5. Guided reproduction

Write a cache decision for three cases: reused expensive aggregate, one-time filter, tiny dimension broadcast.

6. Independent task

Choose one reused expensive lineage and one one-time lineage. Cache/materialize/unpersist only the reused one, leave the one-time lineage uncached, and save evidence supporting both decisions.

7. Validate before trusting

You can state what is cached, why it is reused, how it is materialized, and when it is released.

8. Recover if weak

If caching seems to have no benefit, verify the same lineage is actually reused and compare runtime/storage metrics after materialization.

9. Explain in your own words

Why can caching make a pipeline worse when the dataset is used only once?

10. Evidence to save

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran.

11. Protected Review rule

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

Authoritative current references

[Spark SQL performance tuning - caching](#)

DE12-14 - Parquet I/O and partition pruning

Prerequisite: DE12-13 and M11 storage foundations.

1. Outcome

Read/write Parquet with Spark and prove partition pruning without confusing directory partitions with Spark execution partitions.

2. Teacher explanation

Spark can read and write Parquet while preserving column types and using Parquet metadata. ``partitionBy("region")`` during write creates directory-style dataset partitions. A filter on a partition column can let Spark prune irrelevant directories. This storage partitioning is different from the number of in-memory Spark partitions used by tasks. Always reconcile write/read-back row count and measures; optimization evidence can include FileScan/PartitionFilters in EXPLAIN.

3. Working demonstration

The reference pipeline writes normal orders partitioned by region, then reads back only East and selects `order_id/amount`. Inspect the output directories and formatted plan.

4. Why this matters

Distributed systems can make wrong work faster. The engineer must preserve grain, cardinality, measures and deterministic behavior while also reasoning about partitions, movement, memory and plans.

Video first

[DataTalksClub Zoomcamp - Apache Spark / PySpark](#)

DataFrame I/O visual; M11 knowledge is reused for Parquet/pruning.

DE12-14 - Practice, Proof & Transfer

5. Guided reproduction

Write a second Parquet dataset partitioned by category and compare the files/directories considered for a category filter.

6. Independent task

Write a changed Parquet dataset partitioned by a different low-cardinality column, read back a filtered subset, reconcile full and filtered controls, and inspect PartitionFilters/file-scan evidence for pruning.

7. Validate before trusting

Full read remains 16 rows / 3770.00. East filter returns only East rows. Explain both business and physical partition meanings.

8. Recover if weak

If pruning is absent, verify the filter targets an actual partition column and inspect PartitionFilters rather than guessing from runtime alone.

9. Explain in your own words

What is the difference between a Spark execution partition and a Parquet directory partition?

10. Evidence to save

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran.

11. Protected Review rule

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

Authoritative current references

[Spark Parquet data source](#)

DE12-15 - Built-in functions vs Python UDFs

Prerequisite: DE12-14.

1. Outcome

Prefer Spark built-in expressions over Python UDFs when practical and explain the optimizer/serialization boundary.

2. Teacher explanation

Spark SQL built-in functions are visible to Spark's optimizer and execute within Spark's optimized expression engine. A Python UDF crosses a language/runtime boundary and can add serialization overhead while making expression semantics less transparent to Catalyst. UDFs are still useful when required logic has no suitable built-in; the rule is prefer built-ins, measure before optimizing, and test semantics.

3. Working demonstration

Create an uppercase/trimmed category with built-ins. Then write an equivalent simple Python UDF only for comparison and inspect plans/runtime on a learner machine.

4. Why this matters

Distributed systems can make wrong work faster. The engineer must preserve grain, cardinality, measures and deterministic behavior while also reasoning about partitions, movement, memory and plans.

Video first

[DataTalksClub Zoomcamp - Apache Spark / PySpark](#)

Built-in expression visual; book explicitly compares Python UDF boundary.

DE12-15 - Practice, Proof & Transfer

5. Guided reproduction

Find three transformations in your pipeline that should stay built-in and one hypothetical case that might justify a UDF.

6. Independent task

Without copying the guided example, implement one changed text/conditional transformation using built-in Spark functions and an equivalent Python UDF. Prove equal outputs first, then explain why built-ins remain the default choice.

7. Validate before trusting

Built-in and UDF versions must return equivalent values before any performance comparison.

8. Recover if weak

If you reach for a UDF first, search current `pyspark.sql.functions` for an equivalent expression and compose built-ins where readable.

9. Explain in your own words

Why can a Python UDF be correct yet still be a worse default than a built-in Spark SQL expression?

10. Evidence to save

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran.

11. Protected Review rule

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

Authoritative current references

[PySpark SQL functions](#)

[pyspark.sql.functions.udf](#)

DE12-16 - Driver safety: collect, toPandas and bounded inspection

Prerequisite: DE12-15.

1. Outcome

Protect the driver by bounding local collection and distinguishing debugging samples from distributed processing.

2. Teacher explanation

``collect()`` returns all rows to the driver as Python objects. ``toPandas()`` also materializes data on the driver and should only be used when the result is small enough. Large unbounded collection can exhaust driver memory and defeats distributed processing. Use ``limit``, ``take``, small aggregate outputs, sampled data or writes for inspection. Before collecting, estimate row count/size and justify the bound.

3. Working demonstration

Use ``orders.limit(5).collect()`` for a tiny debug sample. Compare this with the unsafe habit of calling `collect` on an unknown large source.

4. Why this matters

Distributed systems can make wrong work faster. The engineer must preserve grain, cardinality, measures and deterministic behavior while also reasoning about partitions, movement, memory and plans.

Video first

[DataTalksClub Zoomcamp - Apache Spark / PySpark](#)

Driver/distributed-work visual; book adds bounded-inspection rules.

DE12-16 - Practice, Proof & Transfer

5. Guided reproduction

Rewrite three debugging patterns so each has an explicit bound or aggregated result before reaching the driver.

6. Independent task

Rewrite three unsafe inspection requests into bounded driver-safe checks using limit/take/aggregates or writes. For each, state the maximum rows expected to reach the driver and why that bound is acceptable.

7. Validate before trusting

Every local-inspection action states an expected maximum row count and why it is safe on the learner laptop.

8. Recover if weak

If you need “all rows to inspect,” define the question you are debugging and reduce columns/rows first.

9. Explain in your own words

Why is ``limit(20).toPandas()`` a different risk profile from ``large_df.toPandas()``?

10. Evidence to save

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran.

11. Protected Review rule

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

Authoritative current references

[DataFrame.collect](#)

[DataFrame.toPandas](#)

DE12-17 - Testing and debugging Spark transformations

Prerequisite: DE12-16.

1. Outcome

Test Spark transformations with deterministic fixtures and debug failures using schema + row + plan evidence.

2. Teacher explanation

Spark code should be tested as transformations, not only by looking at notebook output. PySpark 4.2 provides testing utilities such as `assertDataFrameEqual` and `assertSchemaEqual`. Keep transformation functions small, feed deterministic fixtures, normalize order where order is not part of the contract, and assert schema plus business controls. For a failure, capture the smallest reproducible input and inspect schema/plan rather than repeatedly rerunning a huge job.

3. Working demonstration

After installing PySpark, run `test_transformations.py`. It builds a tiny `DataFrame` and checks the derived ``net`` column with `assertDataFrameEqual`.

4. Why this matters

Distributed systems can make wrong work faster. The engineer must preserve grain, cardinality, measures and deterministic behavior while also reasoning about partitions, movement, memory and plans.

Video first

[DataTalksClub Zoomcamp - Apache Spark / PySpark](#)

Visual refresher only; testing task is independent and follows current PySpark testing docs.

DE12-17 - Practice, Proof & Transfer

5. Guided reproduction

Write one test for a join that expects no fan-out and one test for a transformation that deliberately preserves null handling.

6. Independent task

Write a fresh deterministic unit test for a transformation not used in the bundled example. Include schema/business assertions and a deliberate failing case; save the failure and corrected pass.

7. Validate before trusting

Tests are deterministic, order-aware where necessary, and fail when the intended contract is broken.

8. Recover if weak

If equality fails for distributed results, first decide whether row order is part of the contract. Sort by a stable key before comparing when appropriate.

9. Explain in your own words

What should a good Spark transformation test prove beyond “the code did not throw an exception”?

10. Evidence to save

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran.

11. Protected Review rule

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

Authoritative current references

[Testing PySpark](#)

DE12-18 - Integrated PySpark pipeline and distributed-processing checkpoint

Prerequisite: DE12-01 through DE12-17.

1. Outcome

Build and defend an integrated local PySpark pipeline with explicit schema, quality controls, joins, aggregation, plan evidence, safe partitioning, Parquet output and tests.

2. Teacher explanation

The module checkpoint combines correctness and execution reasoning. Start with explicit schema and source controls. Transform with built-ins, validate joins before tuning, aggregate and reconcile measures, inspect plans for shuffle/join behavior, write partitioned Parquet, read it back with a pruning filter, and keep driver inspection bounded. Save version/runtime settings. A pipeline is not complete because it ran once: rerun it from a clean output and prove deterministic controls.

3. Working demonstration

Run the reference pipeline only after PySpark 4.2.0 is available. Then rebuild the same requirements from a blank file rather than copying it. Capture explain output and final controls.

4. Why this matters

Distributed systems can make wrong work faster. The engineer must preserve grain, cardinality, measures and deterministic behavior while also reasoning about partitions, movement, memory and plans.

Video first

[DataTalksClub Zoomcamp - Apache Spark / PySpark](#)

Review first-look only; close it before building the integrated pipeline independently.

DE12-18 - Practice, Proof & Transfer

5. Guided reproduction

Build the changed Clinic Shipments retest only if assigned; for normal mastery, construct your own equivalent pipeline over normal orders with one deliberate validation failure injected and repaired.

6. Independent task

From blank files and the normal fixtures, build the full integrated M12 pipeline independently: explicit schema, validation, safe join, aggregation, plan evidence, partition decision, Parquet write/read, bounded inspection and tests. Do not copy the reference pipeline.

7. Validate before trusting

Normal integrated controls: 16 order rows, 31 quantity, 3770.00 amount, 16-row customer join, zero unmatched customer segments, four region groups.

8. Recover if weak

If the integrated job fails, isolate in this order: environment/session -> schema/read -> quality -> join cardinality -> transformation -> action/write -> plan/tuning. Repair the smallest failed layer first.

9. Explain in your own words

In 90 seconds, explain your pipeline from source grain through physical execution and name one correctness failure plus one performance failure that produce different symptoms.

10. Evidence to save

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran.

11. Protected Review rule

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

Authoritative current references

[PySpark getting started](#)

[Spark SQL performance tuning](#)